

Diseño e Implementación de una Base de Datos para Telemetría de Videojuego y Evaluación de Experiencia de Usuario

Beiker Romario Jaramillo Giraldo
Pontificia Universidad Javeriana
Bogotá, Colombia
jaramillo.beiker@javeriana.edu.co

Valentina Carrillo Peñuela
Pontificia Universidad Javeriana
Bogotá, Colombia
carrillo-valentina@javeriana.edu.co

Arturo Enrique Hincapié Orozco
Pontificia Universidad Javeriana
Bogotá, Colombia
arturo-hincapieo@javeriana.edu.co

Abstract—Este documento presenta la segunda entrega del proyecto semestral de Bases de Datos. Se diseña e implementa una base de datos relacional en PostgreSQL para almacenar y analizar telemetría de juego y encuestas de experiencia de usuario (UX) provenientes de una versión modificada de *Chocolate-Doom*. Se describen los cambios realizados respecto a la primera entrega y se documenta el modelo lógico, el pipeline ETL, las consultas analíticas y la estrategia de indexación.

Index Terms—base de datos relacional, telemetría, experiencia de usuario, *Chocolate-Doom*, PostgreSQL

I. CAMBIOS RESPECTO A LA PRIMERA ENTREGA

Durante el desarrollo de la segunda entrega se realizó una revisión del esquema DDL inicial para consolidar el modelo final. El objetivo principal fue adaptar la base de datos al formato real de los datos de telemetría obtenidos desde *Chocolate-Doom* y optimizar el rendimiento de cara al proceso de ingesta masiva en el pipeline ETL.

A. Ajustes de Integridad en la Tabla Player

Se añadió la restricción CHECK ($\text{age} > 0$) sobre el atributo de edad. Esto garantiza la calidad del dato, impidiendo el almacenamiento de edades inválidas o negativas en el perfil de los jugadores.

B. Modificaciones de Unicidad en la Tabla Episode

El campo `episode_number` pasó de ser un entero simple a incorporar la restricción INT UNIQUE NOT NULL. Con esto se evita la duplicación de registros de episodios y se establece una base sólida para las relaciones jerárquicas con mapas y partidas.

C. Reestructuración de la Tabla Game

Se realizaron dos cambios principales:

- **Tipos temporales:** `start_time` TIME y `end_time` TIME fueron reemplazados por `started_at` TIMESTAMP.
- **Identificador de mapa:** Se eliminó `map_code` VARCHAR(10) y se sustituyó por `map_number` INT, reflejando que el archivo de telemetría identifica los mapas mediante valores enteros.

D. Automatización de Integridad en GameParticipant

Se añadieron las cláusulas ON DELETE CASCADE a las llaves foráneas `fk_gp_game` y `fk_gp_player`. Esto garantiza que al eliminar una partida o un jugador, las relaciones se eliminen automáticamente, manteniendo la integridad referencial.

E. Rediseño y Normalización de la Tabla Sector

La estructura original almacenaba sectores mediante cadenas de texto libres (`sector_name`) vinculadas directamente a la partida. Se reemplazó por los atributos `episode_id`, `map_number`, `sector_x` y `sector_y`, con la restricción UNIQUE (`episode_id`, `map_number`, `sector_x`, `sector_y`). Esto elimina la dependencia transitiva del sector respecto a la tabla Game, alcanzando la Tercera Forma Normal (3NF) y evitando redundancia durante la ingesta masiva.

F. Optimización de TelemetryEvent

Al ser la tabla de mayor volumen (un registro por tic), requirió los siguientes cambios:

- **Nuevos atributos:** Se incorporaron `event_timestamp`, `angle`, `momx` y `mom_y` para capturar la estampa de tiempo real, la orientación de cámara y los vectores de momento del jugador.
- **Nomenclatura:** `posi_x`, `posi_y` y `posi_z` se renombraron a `pos_x`, `pos_y` y `pos_z` para unificar el estándar semántico con el script ETL.
- **Restricciones de dominio:** Se implementaron CHECK ($\text{tic} \geq 0$), CHECK ($\text{angle} \geq 0$ AND $\text{angle} \leq 360$), CHECK ($\text{health} \geq 0$) y CHECK ($\text{ammo} \geq 0$) para bloquear la inserción de datos corruptos.
- **Atributo shield:** Se estandarizó con CHECK ($\text{shield} \geq 0$), alineándolo con la columna homónima del log.

G. Evolución de la Tabla UXResponse

Se integró `game_id` INT como clave foránea opcional (admite nulos) hacia la tabla Game. Esto permite correlacionar las respuestas de los instrumentos psicométricos (PENS,

GUESS, BANGS) con los eventos de una sesión específica, añadiendo flexibilidad analítica al modelo.

H. Incorporación de la Tabla *staging_telemetry*

Se introdujo una tabla de *staging* como temporal para los datos crudos del archivo de telemetría antes de su validación y transformación. Todos sus atributos son de tipo TEXT de forma intencional: el objetivo es recibir los datos en su forma más permisiva para detectar errores de formato, valores nulos o fuera de rango antes de la inserción en las tablas normalizadas.

I. Impacto en la Normalización

La eliminación de campos de texto ambiguos (*map_code*, *sector_name*) y el aislamiento lógico de los sectores por mapa erradicaron las dependencias transitivas, consolidando la 3NF y reduciendo los costos de transformación en el pipeline ETL.

II. JUSTIFICACIÓN DE LA NORMALIZACIÓN Y EVOLUCIÓN DEL ESQUEMA RELACIONAL

A. Análisis de Normalización Aplicado al Modelo Final

El diseño del esquema relacional presentado se ha estructurado por las reglas de normalización con el fin de eliminar la redundancia de datos, mitigar anomalías de inserción, actualización y borrado, para optimizar el rendimiento ante la ingesta masiva de datos.

1) *Primera Forma Normal (1NF)*: Todas las tablas del esquema final cumplen con la 1NF. Los atributos contienen únicamente valores atómicos (indivisibles) y no existen grupos repetitivos. Por ejemplo, en *staging_telemetry* el sector se recibe como una cadena compuesta (*sector_x*, *sector_y*), pero en el esquema definitivo esta tupla se descompone y almacena en columnas numéricas atómicas individuales dentro de la tabla *Sector*.

2) *Segunda Forma Normal (2NF)*: Al poseer llaves primarias simples en su mayoría (*player_id*, *episode_id*, *game_id*, *sector_id*, *event_id*), se cumple de forma natural la 2NF, dado que todos los atributos no primos dependen por completo de la clave primaria y no de subconjuntos de esta. Para la tabla de quiebre *GameParticipant*, cuya clave es compuesta (*game_id*, *player_id*), no existen dependencias parciales; la participación requiere la existencia obligatoria de ambas entidades.

3) *Tercera Forma Normal (3NF) – Caso de Estudio Sector*: El rediseño más significativo orientado a la 3NF se aplicó sobre la entidad *Sector*. En arquitecturas previas, el sector se vinculaba como un texto directamente dependiente de la sesión de juego (*Game*). Al reformular la tabla con los atributos *episode_id*, *map_number*, *sector_x* y *sector_y* protegidos por una restricción UNIQUE, se eliminó la dependencia transitiva: De este modo, un sector geométrico existe de forma independiente a si una partida específica lo ha transitado o no, consolidando formalmente la 3NF y previniendo la duplicación exponencial de coordenadas espaciales en la base de datos.

III. PIPELINE DE INGESTA (ETL)

El pipeline ETL se estructura en cuatro fases secuenciales: (1) limpieza y preparación del archivo fuente mediante Python, (2) creación del esquema relacional y la tabla de *staging*, (3) inserción de datos maestros de referencia, y (4) transformación y carga final desde *staging* hacia las tablas normalizadas.

A. Fase 1 – Limpieza del Archivo Fuente (Python)

Antes de cualquier operación en la base de datos, se ejecuta un script en Python que filtra el archivo de log original de *Chocolate-Doom*. Su función es conservar únicamente las líneas que representan eventos de telemetría válidos. El resultado es un archivo *.tsv* limpio, listo para ser copiado a la tabla de *staging*.

```
import re

archivo_entrada = "beiker (1).txtxt"
archivo_salida = "telemetry_clean_for_staging
                .tsv"

with open(archivo_entrada, "r") as entrada, \
    open(archivo_salida, "w") as salida:
    for linea in entrada:
        linea = linea.strip()
        # Solo se conservan lineas que inician
        # con fecha (AAAA-MM-DD)
        if re.match(r"^\d{4}-\d{2}-\d{2}",
                    linea):
            salida.write(linea + "\n")

print("Paso 1 completado: archivo TSV generado
      con exito.")
```

Listing 1. Script de limpieza del archivo de telemetría

B. Fase 2 – Creación del Esquema DDL

Se eliminan las tablas existentes con CASCADE para garantizar un entorno limpio en cada ejecución, y a continuación se recrea el esquema completo. La tabla *staging_telemetry* se define al final del bloque DDL con todos sus campos en tipo TEXT, actuando como zona de amortiguación antes de la validación.

```
-- Limpieza total preventiva de ejecuciones
previas
DROP TABLE IF EXISTS UXResponse          CASCADE
;
DROP TABLE IF EXISTS UXInstrument         CASCADE
;
DROP TABLE IF EXISTS TelemetryEvent       CASCADE
;
DROP TABLE IF EXISTS Sector               CASCADE
;
DROP TABLE IF EXISTS GameParticipant      CASCADE
;
DROP TABLE IF EXISTS Game                 CASCADE
;
DROP TABLE IF EXISTS Episode              CASCADE
;
DROP TABLE IF EXISTS Player               CASCADE
;
```

```

DROP TABLE IF EXISTS staging_telemetry CASCADE
;

-- Tabla de jugadores
CREATE TABLE Player (
    player_id SERIAL PRIMARY KEY,
    nickname VARCHAR(50) UNIQUE NOT
        NULL,
    age INT CHECK (age > 0),
    gender VARCHAR(15),
    experience_level VARCHAR(30)
);

-- Tabla de episodios
CREATE TABLE Episode (
    episode_id SERIAL PRIMARY KEY,
    episode_number INT UNIQUE NOT NULL,
    title VARCHAR(50)
);

-- Tabla de partidas
CREATE TABLE Game (
    game_id SERIAL PRIMARY KEY,
    episode_id INT NOT NULL,
    map_number INT NOT NULL,
    started_at TIMESTAMP,
    ended_at TIMESTAMP,
    CONSTRAINT fk_game_episode
    FOREIGN KEY (episode_id) REFERENCES
        Episode(episode_id)
);

-- Relacion N:M entre partidas y jugadores
CREATE TABLE GameParticipant (
    game_id INT NOT NULL,
    player_id INT NOT NULL,
    PRIMARY KEY (game_id, player_id),
    CONSTRAINT fk_gp_game
    FOREIGN KEY (game_id)
    REFERENCES Game(game_id) ON DELETE CASCADE
,
    CONSTRAINT fk_gp_player
    FOREIGN KEY (player_id)
    REFERENCES Player(player_id) ON DELETE
        CASCADE
);

-- Tabla de sectores del mapa
CREATE TABLE Sector (
    sector_id SERIAL PRIMARY KEY,
    episode_id INT NOT NULL,
    map_number INT NOT NULL,
    sector_x INT NOT NULL,
    sector_y INT NOT NULL,
    UNIQUE (episode_id, map_number, sector_x,
        sector_y),
    CONSTRAINT fk_sector_episode
    FOREIGN KEY (episode_id) REFERENCES
        Episode(episode_id)
);

-- Tabla principal de telemetria (un registro
    por tic)
CREATE TABLE TelemetryEvent (
    event_id SERIAL PRIMARY KEY,
    event_timestamp TIMESTAMP NOT NULL,
    tic INT NOT NULL CHECK (tic

```

```

        >= 0),
    pos_x FLOAT NOT NULL,
    pos_y FLOAT NOT NULL,
    pos_z FLOAT NOT NULL,
    angle FLOAT CHECK (angle >= 0
        AND angle <= 360),
    momx FLOAT,
    momy FLOAT,
    shield INT CHECK (shield >= 0),
    health INT CHECK (health >= 0),
    ammo INT CHECK (ammo >= 0),
    sector_id INT NOT NULL,
    player_id INT NOT NULL,
    game_id INT NOT NULL,
    CONSTRAINT fk_te_sector
    FOREIGN KEY (sector_id) REFERENCES
        Sector(sector_id),
    CONSTRAINT fk_te_player
    FOREIGN KEY (player_id) REFERENCES
        Player(player_id),
    CONSTRAINT fk_te_game
    FOREIGN KEY (game_id) REFERENCES
        Game(game_id)
);

-- Tabla de instrumentos de UX (PENS, GUESS,
    BANGS)
CREATE TABLE UXInstrument (
    instrument_id SERIAL PRIMARY KEY,
    instrument_name VARCHAR(30) NOT NULL
);

-- Tabla de respuestas UX vinculadas
    opcionalmente a una partida
CREATE TABLE UXResponse (
    response_id SERIAL PRIMARY KEY,
    player_id INT NOT NULL,
    instrument_id INT NOT NULL,
    game_id INT,
    score FLOAT,
    responded_at TIMESTAMP DEFAULT
        CURRENT_TIMESTAMP,
    CONSTRAINT fk_ux_player
    FOREIGN KEY (player_id) REFERENCES
        Player(player_id),
    CONSTRAINT fk_ux_instrument
    FOREIGN KEY (instrument_id) REFERENCES
        UXInstrument(instrument_id),
    CONSTRAINT fk_ux_game
    FOREIGN KEY (game_id) REFERENCES
        Game(game_id)
);

-- Tabla de staging: area temporal de carga en
    texto plano
CREATE TABLE staging_telemetry (
    event_timestamp TEXT,
    tic TEXT,
    pos_x TEXT,
    pos_y TEXT,
    pos_z TEXT,
    angle TEXT,
    momx TEXT,
    momy TEXT,
    shield TEXT,
    health TEXT,
    ammo TEXT,

```

```
sector_raw TEXT
);
```

Listing 2. Limpieza preventiva y creación del esquema relacional

C. Fase 3 – Inserción de Datos Maestros

Una vez creado el esquema, se insertan los registros de referencia: los seis jugadores del experimento, los cuatro episodios del juego, la partida registrada y sus participantes. La cláusula ON CONFLICT DO NOTHING garantiza idempotencia en ejecuciones repetidas del pipeline.

```
INSERT INTO Player (player_id, nickname, age,
gender, experience_level)
VALUES
(1, 'Valentina', 23, 'Femenino', '
Avanzado'),
(2, 'Beiker', 21, 'Masculino', '
Experto'),
(3, 'Lisa', 24, 'Femenino', '
Principiante'),
(4, 'Arturo', 22, 'Masculino', '
Intermedio'),
(5, 'Ana', 20, 'Femenino', '
Avanzado'),
(6, 'David', 25, 'Masculino', '
Principiante')
ON CONFLICT (player_id) DO NOTHING;

INSERT INTO Episode (episode_id,
episode_number, title)
VALUES
(1, 1, 'Knee-Deep in the Dead'),
(2, 2, 'The Shores of Hell'),
(3, 3, 'Inferno'),
(4, 4, 'Thy Flesh Consumed')
ON CONFLICT (episode_id) DO NOTHING;

INSERT INTO Game (game_id, episode_id,
map_number, started_at)
VALUES (1, 1, 5, '2026-05-28 12:56:40')
ON CONFLICT (game_id) DO NOTHING;

INSERT INTO GameParticipant (game_id,
player_id)
VALUES (1,1), (1,2), (1,3), (1,4), (1,5), (1,6)
ON CONFLICT DO NOTHING;
```

Listing 3. Inserción de datos maestros de referencia

D. Fase 4 – Transformación y Carga Final

Con el archivo TSV ya cargado en staging_telemetry se ejecutan dos sentencias INSERT ...SELECT: la primera extrae los sectores únicos del campo sector_raw y los registra en la tabla Sector; la segunda realiza todos los CAST de tipo y carga los eventos de telemetría en TelemetryEvent, enlazando cada registro con su sector, jugador y partida correspondientes.

```
INSERT INTO Sector (episode_id, map_number,
sector_x, sector_y)
SELECT DISTINCT
```

```
1, -- episode_id fijo para esta sesion
5, -- map_number fijo para esta sesion
CAST(split_part(TRIM(BOTH '()' FROM
sector_raw), ',', 1) AS INT),
CAST(split_part(TRIM(BOTH '()' FROM
sector_raw), ',', 2) AS INT)
FROM staging_telemetry
ON CONFLICT (episode_id, map_number, sector_x,
sector_y) DO NOTHING;
```

Listing 4. Extracción de sectores únicos desde staging

```
INSERT INTO TelemetryEvent (
event_timestamp, tic,
pos_x, pos_y, pos_z,
angle, momx, momy,
shield, health, ammo,
sector_id, player_id, game_id
)
SELECT
CAST(st.event_timestamp AS TIMESTAMP),
CAST(st.tic AS INT),
CAST(st.pos_x AS FLOAT),
CAST(st.pos_y AS FLOAT),
CAST(st.pos_z AS FLOAT),
CAST(st.angle AS FLOAT),
CAST(st.momx AS FLOAT),
CAST(st.momy AS FLOAT),
CAST(st.shield AS INT),
CAST(st.health AS INT),
CAST(st.ammo AS INT),
sec.sector_id,
1, -- player_id
1 -- game_id
FROM staging_telemetry st
JOIN Sector sec
ON sec.episode_id = 1
AND sec.map_number = 5
AND sec.sector_x = CAST(
split_part(TRIM(BOTH '()' FROM st.sector_raw
), ',', 1) AS INT)
AND sec.sector_y = CAST(
split_part(TRIM(BOTH '()' FROM st.sector_raw
), ',', 2) AS INT);
```

Listing 5. Carga final de eventos de telemetría con conversión de tipos

E. Fase 5 – Poblado de Instrumentos y Respuestas UX

Una vez cargada la telemetría, se procede a registrar los instrumentos de medición de experiencia de usuario y a simular las respuestas de los jugadores. Cada una orientada a capturar un aspecto distinto de la experiencia de juego.

```
('Inmersion Espacial'),
('Desafio / Dificultad'),
('Competencia Percibida'),
('Carga Mental (NASA-TLX)'),
('Satisfaccion General');
```

Listing 6. Registro de dimensiones del instrumento UX personalizado

Con los instrumentos registrados, se generan respuestas simuladas para cada jugador participante en la partida. El producto cartesiano entre GameParticipant y

UXInstrument garantiza que cada jugador tenga una puntuación por cada dimensión. La función `RANDOM()` produce puntajes decimales en el rango `[1.0, 5.0]`, coherente con escalas Likert estándar, redondeados a dos decimales mediante `ROUND(...::numeric, 2)`.

```
INSERT INTO UXResponse (player_id,
    instrument_id, game_id, score)
SELECT
    gp.player_id,
    inst.instrument_id,
    gp.game_id,
    ROUND((1.0 + (RANDOM() * 4.0))::numeric,
        2) AS score
FROM GameParticipant gp
CROSS JOIN UXInstrument inst;
```

Listing 7. Generación de respuestas UX simuladas para todos los jugadores

IV. CONCLUSIONES

En esta segunda entrega se logró consolidar una base de datos relacional capaz de almacenar y gestionar información de telemetría y experiencia de usuario proveniente de Chocolate-Doom. Los ajustes realizados al esquema permitieron mejorar la integridad de los datos, reducir redundancias mediante la normalización y optimizar el proceso de carga a través de un pipeline ETL.

Además, la implementación de la tabla de staging y los procesos de transformación garantizan una carga más segura y eficiente de la información. En conjunto, el sistema desarrollado proporciona una base sólida para realizar análisis posteriores sobre el comportamiento de los jugadores y su experiencia durante las partidas.

REFERENCES

- [1] A. O. Calderon, “trajectory_doom,” GitHub, 2024. [Online]. Available: https://github.com/aocalderon/trajectory_doom